

Transcript

Module 9: Relational Databases and SQL

Video 1: Module Overview

Welcome to the module on relational databases and SQL. SQL and relational data modelling are not just skills but essential tools that empower full stack developers to create dynamic, and scalable web applications. Structured Query Language and data modelling are critical skills for any backend developer. Mastering these skills in your backend development journey will provide a strong foundation for building robust and scalable web applications. Understanding relational databases and mastering database relationships are important in interacting with relational databases which serve as a backbone for storing, organising and retrieving applications data.

Data modelling acts as the architect's blueprint, allowing you to construct a database schema that mirrors the data your application will handle. This ensures the functionality and good user experience in your final product. SQL is the standard language for database interaction. With SQL, you can craft queries that not only retrieve data but also write modified data from the database. This ensures seamless communication between backend components such as the database server and client application.

In this module, we will learn more about relational databases, the relationships between database tables, and how to write SQL queries. We will cover the following topics: Introduction to database, database modelling, organising data relationships using the relational data model, using entity relationship diagram to visualise the data model, and writing basic to more advanced SQL queries. Let us dive in.

Video 2: Introduction to Database Systems

One of the key aspects of backend development is working with databases to store, retrieve, and manipulate data required by the application. A database in the backend development is a structured repository designed to store, manage, and retrieve data for applications. It provides mechanisms for data input, storage, search, retrieval and modification actions.

In the full stack development, data usually originates from the frontend. For example, when a user interacts with a web interface built using HTML, CSS, or JavaScript, and performs actions like adding items to a shopping cart or proceeding to checkout. After which the backend receives this data via HTTP requests from the frontend. In cases where, multiple applications need to access the same database, the backend typically uses a RESTful API. RESTful API defines the URL endpoints and methods such as GET, POST, PUT, DELETE to communicate with the backend database. For example, we can issue a POST request/checkout endpoint with the shopping cart data. These requests are processed by the backend server using database-specific APIs or Object-Relational Mapping (ORM) libraries to store, retrieve or manipulate data within the database.

Relational databases, like MySQL, PostgreSQL and Microsoft SQL are structured into tables. Tables are organised with rows and columns allowing for relationships between different data entities. Relational database supports ACID transactions, prioritising consistency and reliability. ACID is a fundamental concept with the four key properties: Atomicity, Consistency, Isolation, and Durability.

Atomicity – each statement in a transaction to read, write, update or delete data is treated as a single unit. Either the entire statement is executed, or none of it is executed.

Consistency – this ensures that transactions only make changes to tables in predefined predictable ways. Isolation – when multiple users are reading and writing from the same table all at once, isolation of the transactions ensures that the concurrent transactions do not interfere with or affect one another. Durability – ensures that changes to your data made by successful executed transactions will be safe, even in the event of system failure.

Once you have a database set up and an application server connected to it, you need a way to access the data. For relational databases, a language called Structured Query Language SQL, which is pronounced equally as “sequel” has become the ANSI standard for writing queries and manipulating the database. As individual databases have become more complex, SQL has developed many different "dialects" to allow access to some more powerful features that certain databases have adopted. Basic SELECT, INSERT, and UPDATE queries may be transferable between different SQL servers. However, you will often come across cases where you need to write SQL that is specific to the particular database that you are using.

Finally, let us have an overview of the enterprise relational databases landscape which will give you an idea on the type of databases you can choose in your backend development. Some of the most well-known open-source databases include MySQL, SQLite, PostgreSQL and MariaDB. Cloud-based databases, Microsoft Azure, Google Cloud Platform, AWS and IBM DB2 have become increasingly popular as they offer managed services for database maintenance, patching, capacity management, provisioning and infrastructure support. With so many open-source databases to choose from, consider a few factors such as functionality, scalability, support, cost and technical skills when choosing your database.

Think about your full stack development, are you able to specify possible use cases on how the backend database can support the frontend functionalities?

Video 3: Database Modelling and Data Model

Semantic or Conceptual Modelling portrays a business domain at a high level uses concepts familiar to business uses. A well-designed semantic model must have a representation of their domain. What is a data model? A data model is an abstract conceptual model that organises elements of data and standardises how they relate to one another and to the properties of real-world entities.

Let us create a simplistic data model to store registered user's details in an e-commerce web application. First, the database system must have a representation of the objects. The objects in the domain would be our registered users using the e-commerce application the entity types would be the user's profile information, such as addresses and payment methods. An entity represents a particular type of object in the real world. We could represent our registered user as a single entity, the addresses and his or her payment method by two more separate entities, the address and payment method. Attributes describe the characteristics of an entity encapsulating similar attributes and behaviour of the actual physical object.

For example, a "Registered User" entity type would include attributes such as username, password, email, phone, and date joined. An "Address" entity would have attributes such as street, city, country, and postal code. And "Payment Method" entity would have attributes such as payment method, card number and expiry date.

Relationships denote associations between entities. Supposed to prevent scenarios where a user might inadvertently have multiple conflicting addresses or payment methods associated with their profile.

The rule in our relationship ensures that each registered user can have at most one associated payment method and one address. And it defines how the entities will interact within the database.

Business rules are essential in data modelling. They standardise the data view and usage. 1. Facilitate communication between stakeholders and developers. 2. Guide the design process by clarifying data requirements and constraints. 3. Inform the establishment of relationship rules and constraints, ensuring accurate representation of the business processes in the data. 4. Understanding business rules allow us to develop appropriate relationship rules and constraints, and to create an accurate data model. An implementation-ready data model should contain a description of the data structure that will store the end-user data, a set of enforceable rules to guarantee the integrity of the data, a data manipulation methodology to support the applications.

Now, let us install MySQL Community Edition on your system. Visit MySQL Community Downloads and download MySQL Installer. Typically, choose the latest version unless you have specific compatibility requirements. Once the installer executable file is downloaded, double-click to run it. You might be prompted by Windows for administrative privileges. Accept them and proceed with the installation. MySQL Installer provides several setup types. For most users, the "Default Developer" setup type is recommended. Click on "Customise" to adjust the components. Ensure that you have selected 1. MySQL Server, 2. MySQL Workbench, and 3. MySQL Shell. The installer will now download the necessary files and begin the installation process. Follow the prompts to accept the license agreement and configure the server settings using port number 3306 and a secure root password.

Once installation completes, MySQL Server will be configured and started automatically as a Windows service. To verify the installation, open MySQL Workbench and connect to MySQL Server using the root credentials you have set during installation. We have completed the installation of MySQL Workbench and you would be ready for the labs subsequently!

Video 4: Relational Database Model

Let us explore the core relational database concepts. The highest-level logical container is the database which stores all the objects for a common purpose. Next, we have a schema that organises data into separate entities, and common teams. For example, a customer and a sales order table might be defined within a sales schema whereas a product and product order table are placed within a production schema for better consistency. All objects on a database server can be addressed by knowing the database and the schema they reside in, giving us what is known as the four-part name. Server 1 is the name of a database server instance storing the database. In the database, the table order is in the sales schema.

In a relational database, a table generally represents some noun like a customer and its attribute. Each instance of a customer, for example, Dan, Aisha is represented as a single row of data. Attributes are fields that contain a single piece of information such as first name or last name of a customer. To make entities more interesting, we connect them together using the concept of a relationship. Relationships allow us to connect data without having to pull all data into a single table, thus avoiding repeating data. A many-to-many relationship links one instance of an entity with possibly multiple instances of another entity.

Consider, a customer could have placed more than one sales order on different dates. The customer would then have a many-to-many relationship with the SalesOrderHeader table.

Let us examine primary, alternate and foreign keys which play very specific roles within a table. Primary key is a field that uniquely identifies every instance of that entity, for example, unique customers. Every table in your database should have a primary key. All primary key entries are unique and cannot contain null entries. No customers should have a missing or duplicate customer ID as they are uniquely identified by their primary key. If you have more than one key that can perform the entity integrity role after the primary key has been chosen, each remaining candidate key would be referred to as an alternate key.

To establish the relationship between two entities we need to establish a foreign key. For example, if we have a 'Customers' table and a 'SalesOrderHeader' table. Each sales order is linked to a specific customer. Then the 'Customer ID' field in the 'SalesOrderHeader' table should be a foreign key to the 'Customers' table. This ensures that every sales order is associated with a valid customer. To find the purchase dates for sales orders of customer ID 1, you would look at the 'Sales Order Header' table where the 'Customer ID' field matches the value of 1. If order IDs 1 and 3 are associated with customer ID 1 and the dates of this order are the 1st of January and the 1st of February, then a query can confirm that customer ID 1 has indeed placed orders on these dates. This is how foreign key functions to connect data across different tables in a relational database.

The referential integrity states that every non-null foreign key must reference to an existing primary key value. Think about the referential integrity rule, would there be any Sales Order matched with a non-existent customer?

Video 5: Build your database and create an ERD

The Entity Relationship Diagram represents the conceptual database as viewed by the end user modelling simplifies database design and maintenance by enabling you to visualise requirements and resolve design issues. Model-driven database design is an efficient methodology for creating valid databases. MySQL Workbench model editor provides capabilities to 1. create and manipulate a model graphically 2. reverse engineer a live database to a model 3. forward engineer a model to a script or live database and 4. create and edit tables as well as insert data.

We will first build a live database before performing reverse engineer on the database to retrieve our entity relationship diagram. First, begin by establishing a connection to your database server. Click on "Local Connection" to initiate a connection as the root user. Once connected, navigate to the "Schemas" tab to view the list of existing databases. To create a new schema, right-click in the blank area and select "Create Schema". Enter "company" as name of the new schema and click "Apply" to create it. Let us rename "Table Name" field to "department" and create our first entity. Configure a new column, integer "id" primary key with auto increment constraint to our table. Next, continue with the other two attributes, name and location. Click on "Apply" to execute DDL command to create a table in our company schema within your database.

Continue with our second new table employee now. Configure "id" as the integer primary key with auto increment constraint. Let's continue to add new attributes department_id integer, position_id as an integer type, manager_id of integer type, name using VARCHAR (50), characters, salary with DECIMAL (10,2), and date time as hire_date. Complete the last table position Configure id as our integer primary key and add new attributes, which is the unique name and description. MySQL Workbench enables you to add a foreign key from within the table editor. We would also include a foreign key department id in the employee table to reference to the department table.

To do that, right click on Employee Table and select "Alter Table". Navigate to the "Foreign Keys" tab and add the foreign key employee_department.

Remember to set our reference table as department. Next, choose the column `department_id` for "Employee" Table. That will act as the foreign key and select the corresponding column id in the department table to be referenced. For "ON UPDATE" and "ON DELETE" rules, cascades ensures changes or deletions are cascaded through the related tables. We would now include the second foreign key position id in the employee table to reference to the position table. Let us start by adding the foreign key `employee_department` setting our reference table as department. Next, remember to choose the column `position_id` for the employee table that will act as the foreign key.

And last, select the corresponding column id in the department table to be referenced. And for "ON UPDATE" and "ON DELETE" rules, "CASCADE" will ensure changes or deletions cascaded throughout our related tables. In our final foreign key in the employee table, this will self-reference employee table. Right click on employee table and "Alter Table". And define your foreign key name, `employee_position`, and reference table `company.employee`. Our foreign key is `manager_id` and the reference column is `id`.

Now verify that you have built your database tables on your database server. Click on database Select Reverse Engineer. Select your database for which you want to create the Entity-Relationships Diagram, or ER Diagram, before clicking next. After the reverse engineering process gets completed successfully, you will see the ER Diagram of your database. With this knowledge, would you be able to retrieve the Entity Relationship Diagram during the backend development of your database?

Video 6: Write SQL Data Manipulation Language

In this module, we will learn Data Manipulation Language of SQL to make changes to the data inside a relational database. The basic commands of Data Manipulation Language DML, are INSERT which populates tables with new data, UPDATE statement that updates existing data and DELETE statement to remove data from tables. We will learn SELECT statements to help us extract a subset of readable data from the database.

SQL is an ANSI standard supported by most relational database management systems such as MySQL, PostgreSQL and MS SQL Server. Unlike procedural languages like Python or Java, SQL operates as a declarative language where you specify desired outcomes rather than procedural steps. The SQL SELECT statement retrieves data from the database when it is written in a certain syntax. To construct a SELECT statement, begin with the SELECT keyword, followed by placeholder column names. Specify data source using the FROM keyword and end with a semicolon. Another variation of the SELECT statement is often called a "SELECT ALL" or an "Asterisk," which retrieves all of the columns.

To insert data into a relational table, perhaps adding a new customer into our database, structure the INSERT statement in the following way. We will need to specify the number of columns from the table in the first braces and values within the second braces after the INSERT INTO statement Here, value 1 will be added to column 1, value 2 added to column 2, and value n is added to column n. There may be some situations where you will modify data that is already in a table, perhaps a customer who has changed her address. The UPDATE statement allows you to change existing data within a relational table with this syntax. Specify the table name with the keyword UPDATE, and use the SET keyword to specify what is being changed. This could be one or more column names, assigned to a new value with the equality sign.

Use the WHERE clause to filters the rows you want to update. There may be times where you want to delete data from a table, perhaps a customer who has terminated the subscription. We use the SQL DELETE statement to remove rows specified by the WHERE conditions.

Let us execute these SQL statements using the AdventureWorks database. You are given the SQL script to install the necessary tables. To explore the full product table, open a new SQL window using the SQL script tab and initiate a query using SELECT asterisk to retrieve all the columns. Next, specify the name, list price, and standard cost to view these specific fields from your product table. To demonstrate calculations within the SELECT statement, I will add a subtraction operation on ListPrice and StandardCost, and round them off to two decimal places. I will also concatenate the strings colour and size with a comma delimiter. In SQL, an alias is a temporary name assigned to column in the query's result. To make the column's names more readable I will assign an alias markup for the calculated difference and colour size for the concatenated strings.

Execute the query using the 'RUN statement' button to view our results. Suppose, we want to insert a new product category record, with an ID of 99, parent product category 9, and name 'Electric Bike' and date it to the product category table. Our statement will look like this. Specify the five column names and its corresponding values, column names and values should match. Note text values such as 'Electric Bikes' guid are text, and they need to be inside single quotes. And use ANSI CAST function to explicitly convert the string data type into a datetime data type. To delete the previous product category we can use a DELETE statement with a WHERE condition to specify the targeted row with a primary key ID of 99.

We have reviewed the basic SQL statements. Are you now able to write your SELECT INSERT, UPDATE, and DELETE queries on MySQL Workbench to interact with the backend database?

Video 7: Write Advanced SELECT Queries

Earlier, we have looked at all selected or all columns in a table. How can we show only records that meet certain criteria for example, all the orders placed for a particular customer? The WHERE clause allows us to filter data from our query. It can be made up of one or more such conditions which evaluate to TRUE or FALSE. Rows will only be returned when the WHERE clause evaluates as TRUE.

Basic comparison operators such as equal not equal, greater than or equal to and less than or equal to are included in the conditions. We can perform multiple filters on numerical values for example, all records where ID is greater than or equal to 19 and salary is less than 3,000 and the date joined is between 1st January to 31st June.

A regular expression is a powerful way of specifying a pattern for a complex search using the WHERE clause. The search pattern is enclosed by the ^ (hat) at the beginning and dollar operator at the end. Asterisk wildcard matches strings of zeros to any characters and underscore wildcard matches exactly to any one character square brackets matches exactly a single character specified within it and square brackets with a hat symbol excludes any characters specified within it. WHERE firstname, regular Expression ^Ca[m].*n\$. The regular expression functions return names such as Camden Cameron, Camille.

We can also use the GROUP BY keyword which allows us to create groups of data when using aggregate function without having to run separate queries. The HAVING clause will restrict rows in your results after the GROUP BY has been applied and applies to the aggregate function. By understanding the order of processing by the SQL engine database developers can write better performing queries and troubleshoot performance issues.

We will use the regular expression and our WHERE clause to filter product names such as Touring-1000, 2000 and 3000. The query execution of SQL does not follow the order of the SQL statement.

Instead, in a query execution the SQL engine starts by identifying the tables specified in the FROM clause. It determines how these tables will be assessed such as through full table scans or index scans.

Next, the SQL engine filters rows based on the conditions specified in the WHERE clause. Rows that do not satisfy the WHERE conditions are eliminated from further processing. If a GROUP BY clause is present the SQL engine groups the remaining rows into sets based on the colour column specified in the GROUP BY clause. Aggregate functions like AVERAGE is then computed on these group sets. After grouping, the HAVING clause filters the grouped rows based on the specified condition where our average list price is more than \$700. And finally, the SELECT clause computes expressions and functions specified in the SELECT clause including the count and average aggregate functions as well as our case expressions where we have a new product category column.

Finally, our SQL engine retrieves the processed rows and returns them as the query result. With these advanced clauses are you able to write more efficient SELECT queries?

Video 8: Write SQL Joins to combine data from multiple tables

When you design an entire database system using good design principles like normalisation, we often require the use of joins to give a complete picture to a user's query. A join operation allows you to retrieve data from multiple tables in a single SELECT query. Two tables can be joined by a single join operator, but the result can be joined again with other tables. There must exist a same or similar column between the tables being joined.

There are several types of joints, each serving different purposes depending on the data you want to retrieve and how you want to combine it. Inner join: Retrieve rows from both tables where there is a match based on the joint condition. Left join: Retrieves all rows from the left table and the matched rows from the right table. If there is no match, null values are returned for the right table. Right join: is very similar to left joint except all the rows are from the right table. Outer joint: retrieves all rows from both tables and combines them when there is a match. If there is no match, null values are returned.

Let us use the AdventureWorks database. You need to retrieve data about a product and its category from the Product and ProductCategory tables. You can find a relevant product category record for a product based on its ProductSubCategoryID field which is a foreign key in the product table that matches the primary key in the product category table. The FROM clause specifies the product as the left input table. Join operator uses the default INNER JOIN with product category as the right table. The SQL Server will perform a logical Cartesian join on these tables and pass the results as a virtual table. We choose our join key as the product category ID field from the product table. This corresponds to the product category ID field and the product category table with the keyword ON. Using ON clause, SQL Server will filter the virtual table keeping only those rows where a product category ID field from the product table matches a product category ID in the right table. The remaining rows are left in the virtual table and handed off to the next step in the SELECT statement. Finally, the specified column, product name, and category are returned.

Joins can be performed on more than two tables. Suppose you are tasked to analyse June's sales order purchased. You are told to write a query to generate a list of sales order showing the order quantity, product names, and its unit price for each sales order placed in June. The order quantity and unit price is found in the Salesorderdetail table. But the product name is found also within the product table.

And to generate the orders in June, you will need the order date in the sales order header table. We will need to write joins for three tables. We can first start with the sales order header table and perform an inner join with the sales order detail table. Using ON clause, choose SalesOrderID field in both tables as our first join keys. We will perform another join with the product table, choosing product ID field, and both the product and sales order details in the ON clause.

Can you now explain the difference between an inner join and a left join in SQL? When would you then use each type of join?

Video 9: Module Summary

Let us wrap up what we have covered in this module, relational databases and SQL. We started with the basic concepts of database systems and how database can organise and manage structured data efficiently. We looked into database modelling by exploring conceptual, logical, and physical data models. We learned how to translate conceptual designs into physical data models. Next was the relational database model, we understand more about entities, relationships, and primary and secondary keys to ensure entity and referential integrity within our databases.

We learned how to insert, update, and delete data using SQL commands. These are essential for managing dynamic data for practical applications. Advanced select queries took us further, where we write complex queries involving aggregate functions. We also include different clauses such as WHERE, GROUP BY, and HAVING statements. Finally, JOIN statements allow us to combine data from different tables using INNER, LEFT, RIGHT, and SELF joins. This is where things got really powerful, allowing us to fetch exactly the data we need across different related tables.

I hope this module has equipped you with a solid foundation in database systems and SQL. Whether you are aiming to manage data effectively, analyse trends, or build robust applications, these skills will be your toolkit. Keep exploring and applying what you have learned. It is the best way to solidify your understanding and excel in the world of relational databases.

THE END